

EIAN: Explicit Interaction-aware Attention Network for Interpretable Event Modeling

Jiping Zhang*
Huazhong University of Science and
Technology
Wuhan, Hubei, China
jipingz@hust.edu.cn

Hua Zhu*
Huazhong University of Science and
Technology
Wuhan, Hubei, China
zhuhuahust@gmail.com

Hong Huang[†]
Huazhong University of Science and
Technology
Wuhan, Hubei, China
honghuang@hust.edu.cn

Yongkang Zhou
Huazhong University of Science and
Technology
Wuhan, Hubei, China
yongkangzhou_@hust.edu.cn

Kehan Yin[‡]
Huazhong University of Science and
Technology
Wuhan, Hubei, China
yinkehan666@gmail.com

Bang Liu
University of Montreal
Montreal, Quebec, Canada
bang.liu@umontreal.ca

Abstract

Event sequences are integral to domains such as e-commerce, social networks, and healthcare. Traditional point process models, like Poisson and Hawkes processes, are foundational but limited by rigid parametric assumptions, constraining their flexibility in complex real-world scenarios. Neural point processes offer a more adaptable alternative, but typically perform implicit sequence modeling, which does not fully exploit critical event interaction patterns and limits transparency. To address these challenges, we introduce the Explicit Interaction-aware Attention Network (EIAN), a novel model that enhances event modeling by explicitly capturing both intra-type and cross-type event interactions. Specifically, EIAN employs two key components: an intra-type temporal encoder that preserves the unique temporal dynamics within each event type, and a cross-type interaction decoder that highlights interactions across event types. Furthermore, two temporal encoding mechanisms are integrated into the interaction decoder to handle irregular inter-event intervals in diverse temporal scenarios. Extensive experiments show that EIAN consistently outperforms existing models in predictive performance and provides deeper insights into event interaction patterns, advancing both flexibility and interpretability. Our code is available at <https://github.com/CGCL-codes/EIAN.git>.

CCS Concepts

• Information systems → Data stream mining.

*Both authors contributed equally to this research.

[†]Hong Huang is the corresponding author.

[‡]Jiping Zhang, Hua Zhu, Hong Huang, Yongkang Zhou and Kehan Yin are affiliated with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792194>

Keywords

Event Modeling, Interpretability, Point Process

ACM Reference Format:

Jiping Zhang, Hua Zhu, Hong Huang, Yongkang Zhou, Kehan Yin, and Bang Liu. 2026. EIAN: Explicit Interaction-aware Attention Network for Interpretable Event Modeling. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792194>

1 Introduction

Event sequence data are prevalent in various online applications, where understanding interactions among events is essential for accurate predictions and informed decision-making. For example, on e-commerce platforms like Amazon, interactions among customers, sellers, and products generate complex sequences that reflect user behavior and market trends [37]. In online social networks such as Twitter and StackOverflow, user activities—like comments, likes, and shares—form interconnected sequences that capture social influence and information diffusion [18, 43]. Similarly, in healthcare, the temporal patterns between symptoms, diagnoses, and treatments offer critical information on patient care and timely interventions [39]. Therefore, understanding these intricate interactions is crucial.

To model event sequences, temporal point processes provide a robust mathematical framework [28]. The Poisson process [26] assumes that events occur independently at a constant rate, making it suitable for random and uncorrelated events, but limited in capturing dependencies between events. The Hawkes process [11] extends this by incorporating self-exciting mechanisms, where an event can increase the likelihood of subsequent events. This approach has proven useful for applications such as modeling social network activity, financial markets, and aftershocks in seismic activity. Despite its advantages, the Hawkes process lacks flexibility, as it relies on predefined kernel functions and decay rates.

Recent advances in deep learning have led to neural point process models, providing a more flexible framework. These models commonly employ recurrent neural networks (RNNs) [7, 12] or attention mechanisms [31] to learn intricate event dependencies directly from data, eliminating the need for predefined functional forms. For example, [8] and [22] utilize continuous-time RNNs

to dynamically model the intensity function, adapting based on past events. Recent models incorporate self-attention mechanisms [38, 39, 46], which enhance the ability to capture long-term dependencies across sequences, improving model performance in diverse event prediction tasks. In addition, some methods attempt to utilize neural differential equations (NDEs) for event modeling [6, 40].

Although neural point processes have demonstrated impressive results in various real-world applications [36], most still fall short in fully leveraging and explaining the underlying event interaction patterns. Unlike traditional point process models, such as the Hawkes process, which explicitly captures the influence of past events on future occurrences using triggering kernels and excitation matrices, current neural point processes generally employ an implicit sequence modeling. Specifically, these models often compress all historical event information into a hidden representation centered on the latest event, used to predict future events [22, 46]. This approach is largely inspired by sequence modeling techniques in natural language processing (NLP) [5, 31], where future tokens are predicted based on the hidden representation of the latest token.

However, such a paradigm overlooks critical distinctions between event sequences and language sequences. In NLP, words within the same context are often semantically interconnected, whereas in event sequences, a future event might be influenced by only a few past events of certain types, with others being irrelevant [23, 41]. Consequently, this implicit approach may fail to accurately capture event dependencies, thus limiting the model's ability in event modeling. Additionally, the black-box neural networks are limited in interpretability, making it challenging to discern the underlying event interaction patterns. Therefore, while traditional Hawkes processes are widely-used to reveal event interactions and even infer causal relationships [27, 35], neural point processes have seen limited application in such tasks due to the difficulties in extracting interpretable interaction insights from implicit models. Recent efforts, such as [23], attempt explicit interaction modeling, though often at the expense of simplifying the attention mechanism.

To address the challenges, we propose an Explicit Interaction-aware Attention Network (EIAN), which simultaneously overcomes the flexibility issues of traditional point processes and interpretability issues of neural point processes. The first key innovation in EIAN is the *intra-type temporal encoder*, which captures the unique temporal dynamics of each event type independently. To achieve this, the encoder incorporates a type-independent self-attention module that uses a special intra-type mask, ensuring each event attends only to prior events of the same type as itself. This isolates intra-type temporal patterns, preserving type-specific dynamics and preventing cross-type information mixing for interpretability.

Complementing the encoder, the *cross-type interaction decoder* explicitly models interactions between different type events. In this module, type-wise query vectors are assigned to each event type, facilitating the instance-to-type cross-attention between these vectors and the outputs of the temporal encoder. This approach aggregates relevant historical information across event types, generating type-wise state representations at each timestamp and directly capturing the nuanced influence of past events on potential future events of each type, thereby enhancing interpretability.

Specially, to handle irregular inter-event time intervals in diverse temporal scenarios, we incorporate two temporal encoding

mechanisms into the cross-attention module. While the vanilla query vectors are suitable for most settings, it degrades in settings where the underlying event interactions are sensitive to the change of inter-event time intervals. Therefore, we further introduce the time-encoded queries with a *rotary timestamp encoding*, which explicitly encodes irregular inter-event time intervals, allowing a more flexible adaption to event timing dynamics. By enriching the model's temporal representations, this design ensures a more precise capture of the underlying event interactions.

Moreover, leveraging the explicit cross-type attention weights, EIAN can recover an interpretable type-to-type influence matrix from raw event sequences without any supervision. Extensive qualitative and quantitative analyses on both real-world and synthetic datasets demonstrate that the recovered interaction patterns align closely with ground-truth causal graphs, validating that EIAN's predictions are not only accurate but also explainable.

The main contributions of this study are listed as follows:

- We propose a novel model that explicitly captures event interactions, providing both enhanced flexibility and interpretability for understanding these interactions.
- We incorporate two temporal encoding mechanisms for the cross-type interaction decoder, enabling us to handle scenarios with diverse inter-event interval distributions.
- We conduct extensive experiments on seven real-world datasets, demonstrating the superior predictive performance of the proposed model.
- We evaluate the interpretability of our model through both qualitative and quantitative analyses, highlighting its capacity to reveal the underlying event interaction patterns.

2 Related Work

2.1 Statistical Temporal Point Processes

Point processes provide a powerful framework for capturing the occurrence and interaction patterns of asynchronous event sequences [28]. The Poisson process [26], which assumes events occur independently at a constant rate, is suitable for scenarios involving random, uncorrelated events. However, it falls short in representing the dependencies that commonly exist between events in real-world settings, where an event's occurrence is often influenced by previous events. The Hawkes process [11] addresses this limitation by introducing the concept of self-excitation, allowing an event to increase the likelihood of subsequent occurrences. Widely applied to systems such as social networks, financial markets, and earthquake aftershocks [25], the Hawkes process effectively models self-excitatory behaviors. However, its flexibility is limited by the use of predefined kernel functions and constant decay rates, which impose strong parametric assumptions on the model. Other methods, e.g., the self-correcting process [14], in contrast, focuses on inhibitory effects, where past events suppress future events.

2.2 Neural Temporal Point Processes

However, the simplified assumptions about statistical temporal point processes' complex dynamics come at the expense of practical utility. To overcome these constraints, neural point processes [36], leveraging neural networks, have been proposed for greater

adaptability. This line of models typically utilizes recurrent neural networks (RNNs) [7, 12] or attention mechanisms [31] to conduct sequence modeling and learn complex temporal dependencies directly from data, bypassing the need for predefined functional forms. For instance, the Recurrent Marked Temporal Point Process (RMTTP) [8] and Neural Hawkes Process (NHP) [22] use continuous-time RNNs to dynamically model intensity functions based on past events. But the ability of RNN-based methods to capture long-term dependencies is worrying. Aiming to bridge this gap, recent advances incorporate self-attention mechanisms [38, 39, 46], demonstrating advantages in capturing long-term event dependencies. Other extensions use reinforcement learning [19], graphical event models [4], and neural differential equations [6, 16, 40] to enhance model capabilities and prediction accuracy. However, despite advancements in flexibility, these models are limited in interpretability due to the black-box nature of neural networks. Although some studies have recognized this issue and attempted to enhance model interpretability—for instance, the ITHP [23] integrates event time and type information to endow its predictive parameters with practical significance and another method [20] decomposes self-triggering kernels to characterize past events' excitation—they still fall short of explaining the core mechanism: the underlying event interactions that are crucial to sequence prediction.

3 Preliminary Knowledge

This section briefly reviews the Multivariate Hawkes Process (MHP) [11] and the Transformer Hawkes Process (THP) [46], highlighting their fundamental differences in modeling event interactions.

3.1 Multivariate Hawkes Process

The multivariate Hawkes process is a self-exciting point process that models multiple types of events. Its conditional intensity function for event type k at time t is defined as:

$$\lambda_k(t|\mathcal{H}_t) = \mu_k + \sum_{t_i < t} \phi_{kik}(t - t_i), \quad (1)$$

where $\mathcal{H}_t$ denotes the historical events before t , μ_k is the baseline intensity for type k , and ϕ_{kik} is the excitation kernel from type k_i to type k . A common exponential kernel is:

$$\phi_{kik}(\Delta t) = \alpha_{kik} e^{-\beta_{kik} \Delta t}, \quad (2)$$

with $\alpha_{kik} > 0$ and $\beta_{kik} > 0$. This kernel function explicitly describes the interaction patterns between events, thus enabling good interpretability.

3.2 Transformer Hawkes Process

The Transformer Hawkes Process extends the MHP by incorporating the self-attention mechanism for improved flexibility. Its conditional intensity function for a type k event at time t that satisfies $t \in (t_i, t_{i+1}]$ can be expressed as:

$$\lambda_k(t|\mathcal{H}_t) = f_k\left(\alpha_k \frac{t - t_i}{t_i} + \vec{w}_k \circ \vec{h}_{t_i} + b_k\right) \quad (3)$$

where $f_k(x)$ is the softplus function and $\circ$ denotes dot-product. $\vec{h}_{t_i}$ is the hidden representation generated by a self-attention module over the history before t_i , and $\vec{w}_k$ is a type-specific vector.

Unlike the MHP, which uses explicit kernels as defined in Eq(2), this model actually conducts an indirect implicit modeling of each historical event's influence on a type- k event through the interaction between the query vector $\vec{w}_k$ and the hidden vector $\vec{h}_{t_i}$. This approach integrates all historical information into a single hidden vector before influencing the intensity, which poses two issues: (1) the inability to directly model the influence of individual historical events, potentially leading to inaccurate dependency capture; (2) limited interpretability due to the implicit, black-box nature of the modeling process.

4 Methods

4.1 Model Framework

The model framework is illustrated in Figure 1. EIAN comprises two main modules: an intra-type temporal encoder and a cross-type interaction decoder. In the first stage, the intra-type temporal encoder captures temporal dynamics within each event type independently through an intra-type self-attention module, where an intra-type mask restricts each event to attend only to prior events of the same type, preventing cross-type information mixing at this stage (Section 4.2). In the second stage, the cross-type interaction decoder aggregates relevant historical information for each event type by performing instance-to-type cross-attention between the encoder's outputs and type-wise query vectors, generating type-wise state representations for each event type (Section 4.3). Finally, these state representations are passed through three linear predictors to estimate the attributes of the next event (Section 4.4).

4.2 Intra-type Temporal Encoder

In empirical event sequences, instances of the same type typically exhibit stable temporal evolution patterns (e.g., user repurchasing, disease relapse), whereas cross-type influences are sparser and more heterogeneous. If a uniform attention mechanism is naively applied to the entire history, the resulting weights are diluted by high-frequency yet irrelevant events, preventing the model from identifying which types genuinely affect the occurrence of a target event. To remedy this, we first model the intra-type temporal dynamics of each event class independently. An intra-type attention mechanism distills a type-level evolutionary representation that furnishes a clean and compact historical summary for subsequent cross-type influence modeling.

Formally, given an event sequence with L events, $S = \{(t_i, z_i)\}_{i=1}^L$, where t_i and z_i denote the timestamp and type of the i -th event, and $Z = [\vec{z}_1, \vec{z}_2, \dots, \vec{z}_L]$ denoting the one-hot vector of each event's type. We define the initial embedding for the events as $X = (X_t + X_z)^\top \in \mathbb{R}^{L \times D}$, where X_t and X_z represent the timestamp embedding matrix and type embedding matrix. Specifically, $X_t = [\vec{t}_1, \vec{t}_2, \dots, \vec{t}_L] \in \mathbb{R}^{D \times L}$ is a predefined embedding matrix for event timestamps. We adopt the positional encoding from the original Transformer [31] and apply it to encode continuous timestamps. $X_z = W_z Z \in \mathbb{R}^{D \times L}$ and W_z is a trainable embedding matrix for event types.

The initial event embeddings $X = [\vec{x}_1, \vec{x}_2, \dots, \vec{x}_L]$ are then fed into a type-independent self-attention module to capture the temporal evolution patterns for events of each type. This generates a hidden representation for each event as follows:

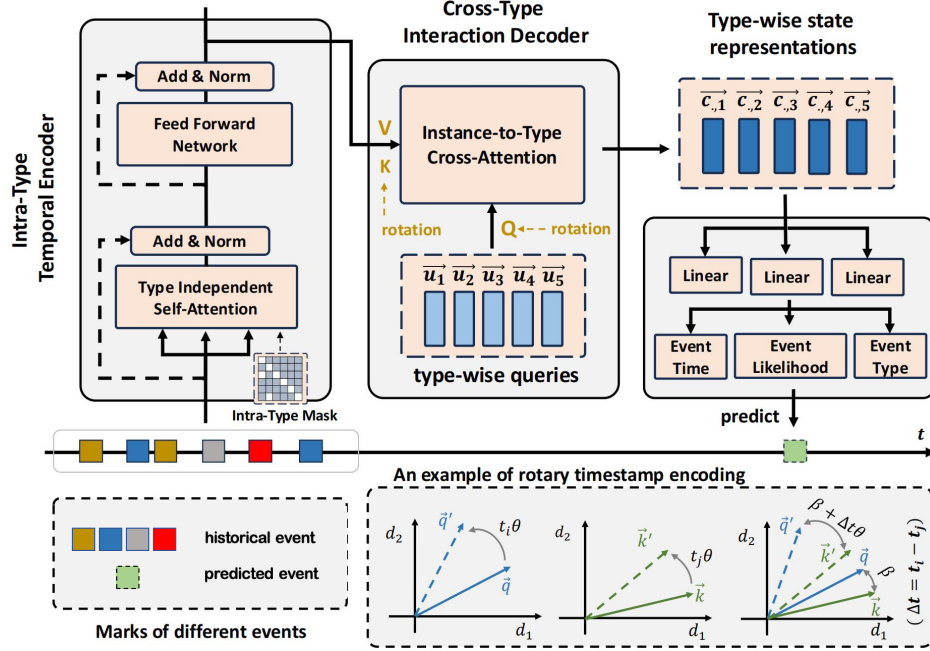


Figure 1: Model framework of our proposed model.

$$\tilde{h}_i = \frac{\sum_{j=1}^i (S_{enc}(\tilde{x}_i, \tilde{x}_j) G_{enc}(\tilde{x}_j))}{\sum_{j=1}^i S_{enc}(\tilde{x}_i, \tilde{x}_j)} \in \mathbb{R}^{D_v} \quad (4)$$

where $G_{enc}(\tilde{x}_j) = W_{enc}^V \tilde{x}_j \in \mathbb{R}^{D_v}$ is a linear transformation that generates value($\mathbf{v}$) in the attention terminology [31]. $S_{enc}(\tilde{x}_i, \tilde{x}_j)$ denotes the similarity between the i -th and j -th events as follows:

$$S_{enc}(\tilde{x}_i, \tilde{x}_j) = \exp\left(\frac{\tilde{q}_i \circ \tilde{k}_j}{\sqrt{D_k}}\right) \times M_{ij} \quad (5)$$

where $\tilde{q}_i = W_{enc}^Q \tilde{x}_i \in \mathbb{R}^{D_k}$, $\tilde{k}_j = W_{enc}^K \tilde{x}_j \in \mathbb{R}^{D_k}$. Specifically, $M \in \mathbb{R}^{L \times L}$ is a carefully designed intra-type mask matrix, where $M_{ij} = 1$ if and only if the j -th event is of the same type as the i -th event and occurs ahead of the i -th event. This ensures that each event can only attend to prior events that share the same type with itself, thereby preventing cross-type information mixing and allowing this module to focus on modeling the temporal dynamics of each event type. Then, the type-independent self-attention module is followed by a position-wise feed-forward neural network (FFN) as follows:

$$\tilde{e}_i = \max(0, \tilde{h}_i W_1 + \tilde{b}_1) W_2 + \tilde{b}_2 \in \mathbb{R}^D \quad (6)$$

where $W_1 \in \mathbb{R}^{D_v \times D_f}$, $W_2 \in \mathbb{R}^{D_f \times D}$, $\tilde{b}_1 \in \mathbb{R}^{D_f}$, and $\tilde{b}_2 \in \mathbb{R}^D$. Intuitively, this vector aggregates all historical events of the same type as the i -th event up to time t_i , distilling the intrinsic temporal evolution patterns of that specific type. By first purifying the historical trajectory within the same type, we furnish a clean and focused contextual representation for subsequent cross-type influence modeling.

In practice, we stack multiple encoder blocks with multi-head attention to enhance model capacity. As shown in Figure 1, each block includes a type-independent self-attention module and an FFN module, with residual connections and layer normalization around both. It is worth mention that, the diagonal of the intra-type mask matrix M is set to 0, as the residual already encodes each event's own information.

4.3 Cross-Type Interaction Decoder

Through the intra-type temporal encoder, we have captured the temporal dynamics for historical events of each type and obtained the hidden representation of each event. In this section, we will capture these historical event instances' influence on future events of different types, aggregating the correlated historical information for each type into type-wise state representations (Figure 1). Specifically, to adapt to real-world scenarios with different inter-event time interval importance, we devise two instance-to-type cross-attention settings with different temporal encoding mechanisms.

4.3.1 Definition of Type-wise Query Vectors. An event occurrence is usually caused by historical event instances of specific types, and each event type corresponds to a distinct influencing set of event types [9]. To explicitly model the influence weight of a historical event type on a future event type, we introduce a learnable *influence query vector* for each target type:

$$\tilde{u}_z = P_\theta[z, :] \in \mathbb{R}^D, \quad z = 1, \dots, N \quad (7)$$

where $P_\theta \in \mathbb{R}^{N \times D}$ is a trainable parameter matrix, N denotes the total number of event types, and $\tilde{u}_z$ is the query vector for event type- z . Unlike the instance-level type embedding X_z in Section 4.2, the vector $\tilde{u}_z$ is a type-level, learnable parameter whose sole

purpose is to quantify how a historical event instance may affect event type z . Although both are type-generated embedding vectors, their parameters are completely independent; this prevents the query vector from inheriting any selection bias induced by the embedding space of instance type and allows it to flexibly capture arbitrary cross-type interactions. The resulting instance-to-type attention yields a direct type-to-type influence matrix, yielding substantially improved interpretability.

4.3.2 Cross-Attention with Vanilla Query. We first introduce the vanilla query setting, which assumes that the strength of influence between events is independent of absolute time, determined solely by event types and their historical context. To consider the influence of historical events, we define the influencing weight of an arbitrary historical event to a subsequent type- z event as follows:

$$S_{dec}(\vec{u}_z, \vec{e}_j) = \exp(\phi(\vec{u}_z, \vec{e}_j)) \quad (8)$$

where $\phi(\vec{u}_z, \vec{e}_j)$ denotes the similarity between the hidden representation of the j -th historical event and the query vector of event type- z , which can be expressed as follows:

$$\phi(\vec{u}_z, \vec{e}_j) = \tanh(W_{dec}^Q \vec{u}_z \circ W_{dec}^K \vec{e}_j) \quad (9)$$

where $W_{dec}^Q \in \mathbb{R}^{D_k \times D}$, $W_{dec}^K \in \mathbb{R}^{D_k \times D}$. Then, we employ these influencing weights as logits, and conduct cross-attention among the historical event instances and each event type to generate type-wise state representations as follows:

$$\vec{c}_{i,z} = \frac{\sum_{j=1}^i (S_{dec}(\vec{u}_z, \vec{e}_j) G_{dec}(\vec{e}_j))}{\sum_{j=1}^i S_{dec}(\vec{u}_z, \vec{e}_j)} \in \mathbb{R}^{D_o} \quad (10)$$

where $G_{dec}(\vec{e}_j) = W_{dec}^V \vec{e}_j \in \mathbb{R}^{D_o}$ is a linear transformation for the j -th historical event's hidden representation. $\vec{c}_{i,z}$ is the state representation for event type- z at t_i , which encodes the relevant historical information that influences the occurrence probability of a type- z event in the future. This vanilla query setting remains adequate for event-intensive sequences whose inter-event gaps are narrowly distributed, yielding fewer parameters and more stable training.

4.3.3 Cross-Attention with Time-Encoded Query. However, when intervals follow an irregular distribution, merely knowing “whether it happened” is no longer sufficient to capture timeliness. To enhance the proposed model's ability to model irregular inter-event time intervals, we adopt the rotary position encoding (RoPE) [30] from the field of NLP, and implement a rotary timestamp encoding to explicitly inject the irregular interval information into the cross-attention procedure by modifying Eq(9) as:

$$\begin{aligned} \phi(\vec{u}_z, \vec{e}_j, \Delta t_{ij}) &= \tanh\left((\mathcal{R}_{\theta, t_i}^{D_k} W_{dec}^Q \vec{u}_z) \circ (\mathcal{R}_{\theta, t_j}^{D_k} W_{dec}^K \vec{e}_j)\right) \\ &= \tanh\left((W_{dec}^Q \vec{u}_z)^\top (\mathcal{R}_{\theta, t_i - t_j}^{D_k}) (W_{dec}^K \vec{e}_j)\right) \end{aligned} \quad (11)$$

where $W_{dec}^Q \vec{u}_z \in \mathbb{R}^{D_k}$, $W_{dec}^K \vec{e}_j \in \mathbb{R}^{D_k}$, and $\Delta t_{ij} = t_i - t_j$ is the interval between the j -th event and the current timestamp t_i . $\mathcal{R}_{\theta, t_i}^{D_k} \in \mathbb{R}^{D_k \times D_k}$ and $\mathcal{R}_{\theta, t_j}^{D_k} \in \mathbb{R}^{D_k \times D_k}$ are two rotary matrices defined as:

$$\mathcal{R}_{\theta, t}^{D_k} = \begin{pmatrix} \cos t\theta_1 & -\sin t\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin t\theta_1 & \cos t\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos t\theta_2 & -\sin t\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin t\theta_2 & \cos t\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos t\theta_{d/2} & -\sin t\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & \sin t\theta_{d/2} & \cos t\theta_{d/2} \end{pmatrix} \quad (12)$$

where t denotes a timestamp value, and $\theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\}$ are pre-defined parameters. Compared to the original Eq(9), $\phi(\vec{u}_z, \vec{e}_j, \Delta t_{ij})$ injects temporal information into both the hidden vectors of historical events and type-wise query vectors via rotation matrices. Specifically, $\mathcal{R}_{\theta, t_i - t_j}^{D_k} = (\mathcal{R}_{\theta, t_i}^{D_k})^\top \mathcal{R}_{\theta, t_j}^{D_k}$ encodes the time interval between the j -th historical event and the current timestamp t_i . Figure 1 gives an example of rotary timestamp encoding. Then, we can inject the inter-event time interval information into Eq(8) and Eq(10), thus modifying them as follows:

$$S_{dec}(\vec{u}_z, \vec{e}_j, \Delta t_{ij}) = \exp(\phi(\vec{u}_z, \vec{e}_j, \Delta t_{ij})) \quad (13)$$

$$\vec{c}_{i,z} = \frac{\sum_{j=1}^i (S_{dec}(\vec{u}_z, \vec{e}_j, \Delta t_{ij}) G_{dec}(\vec{e}_j))}{\sum_{j=1}^i S_{dec}(\vec{u}_z, \vec{e}_j, \Delta t_{ij})} \in \mathbb{R}^{D_o} \quad (14)$$

4.4 Model Training

There are three main tasks that various event modeling methods aim to address. While the maximum likelihood estimation task seeks to maximize the log-likelihood of the entire event sequence through modeling the conditional intensity function, the event time prediction task and the event type prediction task aim to predict the timestamp and type of the next event, respectively.

4.4.1 Maximum Likelihood Estimation. Given a sequence with L events: $S = \{(t_i, z_i)\}_{i=1}^L$, and the state representation for event type- z at t_i denoted as $\vec{c}_{i,z}$, we define the conditional intensity function for a type- z event at $t \in (t_i, t_{i+1}]$ as:

$$\lambda_z(t|\mathcal{H}_t) = f_z(\alpha_z \frac{t - t_i}{t_i} + \vec{w}_\lambda \circ \vec{c}_{i,z} + b_z) \quad (15)$$

where $\mathcal{H}_t = \{(t_j, z_j)\}_{j=1}^i$ denotes all the historical events that happened before t . $f_z(x) = \beta_z \log(1 + \exp(x/\beta_z))$ is the softplus function with a softness parameter $\beta_z > 0$ to ensure the intensity being positive. α_z controls the importance of the interpolation between the current timestamp t_i and the future timestamp t , and b_z represents a type- z event's occurrence probability without considering any history. $\vec{w}_\lambda$ is a shared linear layer among different event types. Here, we adopt a similar formulation to [46], but with the original hidden vector replaced by our type-wise event state representations. Then, the conditional probability density function of a type- z event at $t \in (t_i, t_{i+1}]$ can be expressed as follows:

$$p_z(t|\mathcal{H}_t) = \lambda_z(t|\mathcal{H}_t) \times \exp\left(-\int_{t_i}^t \lambda(\tau|\mathcal{H}_\tau) d\tau\right) \quad (16)$$

where $\lambda(t|\mathcal{H}_t) = \sum_{z=1}^N \lambda_z(t|\mathcal{H}_t)$ summarizes the intensity values of all event types at t . Finally, the log-likelihood for the whole event sequence S can be formulated as:

$$\mathcal{L}_{mle} = \sum_{i=1}^{L-1} \log \lambda_{z_{i+1}}(t_{i+1} | \mathcal{H}_{t_{i+1}}) - \int_{t_1}^{t_L} \lambda(\tau | \mathcal{H}_\tau) d\tau \quad (17)$$

Intuitively, the logarithmic term should be maximized to explain the occurrence of a type- z_{i+1} event at t_{i+1} under the influence of historical events up to t_i , while the integral term should be minimized to explain the absence of any event during the time interval (t_i, t_{i+1}) . Detailed mathematical derivations of the conditional probability density and log-likelihood functions can be found in [28].

4.4.2 Event Timestamp Prediction. To estimate the timestamp of the next event, we first process the concatenation of the type-wise state representations at t_i , i.e., $[\vec{c}_{i,\cdot}]$, through a type-specific linear layer to obtain the estimand for each event type as $\hat{t}_{i+1,z} = \vec{w}_{t,z} \circ [\vec{c}_{i,\cdot}] + \delta_z$, where $\vec{w}_{t,z} \in \mathbb{R}^{D_v}$ is the weight matrix of the linear layer, and δ_z is the bias term for event type z . After obtaining the estimated timestamps $\{\hat{t}_{i+1,z}\}_{z=1}^N$ for each type, we take the minimum one as the proposed model's overall prediction: $\hat{t}_{i+1} = \min(\hat{t}_{i+1,1}, \hat{t}_{i+1,2}, \dots, \hat{t}_{i+1,N})$. Then, the timestamp loss is computed by minimizing the squared error between the prediction and the ground-truth as follows:

$$\mathcal{L}_{time} = \sum_{i=1}^{L-1} (\hat{t}_{i+1} - t_{i+1})^2 \quad (18)$$

4.4.3 Event Type Prediction. Similarly, $[\vec{c}_{i,\cdot}]$ is first passed through a type-specific linear layer to generate the unnormalized logit score for each type as $z_{i+1,z} = \vec{w}_{p,z} \circ \vec{c}_{i,\cdot} + \xi_z$, where $\vec{w}_{p,z} \in \mathbb{R}^{D_v}$ and ξ_z are the linear layer's weight matrix and bias term for event type- z . The logits are then fed into a softmax function to get the category distribution: $\hat{p}_{i+1,k}$. Then, the cross-entropy loss is computed by comparing $\hat{p}_{i+1,k}$ with the ground-truth label y_{i+1} as:

$$\mathcal{L}_{type} = - \sum_{i=1}^{L-1} \sum_{k=1}^N y_{i+1,k} \log \hat{p}_{i+1,k} \quad (19)$$

Finally, the overall optimization loss for the proposed model can be formulated as follows:

$$\mathcal{L} = -\gamma_1 \cdot \mathcal{L}_{mle} + \gamma_2 \cdot \mathcal{L}_{time} + \gamma_3 \cdot \mathcal{L}_{type} \quad (20)$$

4.5 Recovering event interaction patterns

Based on the instance-to-type cross-attention module, we can uncover the underlying event interaction patterns from two distinct perspectives: (1) deriving normalized cross-attention map for each dataset based on the cross-attention scores. (2) intervening the cross-attention procedure through a causal graph for explanation, which can be optimized together with the proposed model within the variational inference [13] framework. Due to the page limit, we provide the details in Appendix A.1 and A.2.

5 Experiments

We adopt 7 real-world datasets to evaluate the models: (1) **Amazon** [24] records product review behaviors of users on the Amazon platform. (2) **Retweet** [45] records retweet behaviors of users on the Twitter platform. (3) **Taxi** [32] includes time-stamped taxi pick-up and drop-off event sequences across five boroughs of the New

York City. (4) **Taobao** [37] collects user click behaviors on Taobao shopping pages. (5) **StackOF** [18] contains user reward histories from the StackOverflow platform. (6) **IPTV** [21] includes time-stamped users' TV watching behavior in internet protocol television systems. (7) **MIMIC-II** [29] collects clinical visits of patients from an electric medical record system. Detailed statistics of each dataset can be found in Appendix B.1.

We compare the proposed model against 7 competitive baselines. Two RNN-based models: RMTTP [8] and NHP [22]. Three Attention-based models: THP [46], AttNHP [38] and ITHP [23]. Two NDE-based models: ODETPP [6] and NJDTPP [40]. Based on the cross-attention settings, we divide the proposed model into two versions: **EIAN-v1**, which uses vanilla queries (Section 4.3.2), and **EIAN-v2**, which explicitly injects irregular event intervals into the cross-attention procedure through the rotary timestamp encoding (Section 4.3.3).

We adopt 3 standard metrics to evaluate the performance of each model from 2 aspects. (1) **Next event prediction**: we adopt the accuracy (ACC) for event type prediction, and the root mean square error (RMSE) for event timestamp prediction. (2) **Goodness-of-fit**: we use the mean log-likelihood (MLL) for each event to conduct goodness-of-fit evaluation. A higher MLL indicates a better goodness-of-fit.

5.1 Overall Performance

5.1.1 Next event prediction. Table 1 presents the results on 7 real-world datasets. For the **event type prediction task**, our model demonstrates superior performance on most of the datasets. In particular, the accuracy improvements on the Amazon and Retweet datasets exceed 14.37% and 17.47%, respectively. Among the baseline models, the two RNN-based ones, i.e., RMTTP and NHP, show similar results on most datasets. In terms of attention-based models, THP achieves the best performance across most datasets. In contrast, ITHP, which simplifies the attention mechanism to model explicit interactions, comes at the cost of performance. Furthermore, as its official implementation does not support next timestamp prediction, we only report the accuracy for ITHP. In addition, another attention-based model AttNHP achieves competitive performance across different datasets, but suffers from out-of-memory (OOM) issues on the IPTV dataset, which stems from its customized attention mechanism. For the **event timestamp prediction task**, our model exhibits significant advantages over the baselines except on Taobao. Notably, our model achieves significant improvements on the Amazon and Retweet datasets.

5.1.2 Goodness-of-fit. We also evaluate the proposed model from the aspect of goodness-of-fit, which assesses how well a model fits the event distribution of the observed event sequences. We compare our model with the state-of-the-art baseline model THP on 7 datasets. Both models use a similar conditional intensity function, thus enabling a fair network architectural comparison. As shown in Table 2, our model achieves superior MLL values on all datasets, demonstrating its effectiveness in modeling event distributions. Besides, we observe that the MLL rankings of these models on each dataset are highly related to their performance on event prediction tasks (Table 1), which means these three tasks and metrics consistently reflect the model performance from different aspects.

Table 1: EIAN’s next-event prediction performance on 7 real-world datasets. For each dataset, the best and second-best results are highlighted in bold and underlined, respectively. Higher ACC and lower RMSE indicate better performance. The results reported are averaged over 5 independent runs with different initialization seeds.

Dataset	Metric	Model								
		RMTPP	NHP	ODETPP	THP	AttNHP	NJDTTP	ITHP	EIAN-v1	EIAN-v2
Amazon	ACC	0.2731 $\pm$ 0.0177	0.2660 $\pm$ 0.0024	0.3249 $\pm$ 0.0082	0.3508 $\pm$ 0.0021	0.2861 $\pm$ 0.0408	0.3412 $\pm$ 0.0010	0.3195 $\pm$ 0.0009	0.4945 $\pm$ 0.0017	<u>0.4895</u> $\pm$ 0.0037
	RMSE	0.6196 $\pm$ 0.0002	0.6208 $\pm$ 0.0001	0.6122 $\pm$ 0.0003	0.6210 $\pm$ 0.0000	1.7576 $\pm$ 1.1978	0.3416 $\pm$ 0.0004	-	<u>0.2948</u> $\pm$ 0.0084	0.2648 $\pm$ 0.0099
Retweet	ACC	0.5462 $\pm$ 0.0051	0.5952 $\pm$ 0.0043	0.5553 $\pm$ 0.0655	0.4917 $\pm$ 0.0964	0.5877 $\pm$ 0.0077	0.5596 $\pm$ 0.0516	0.5697 $\pm$ 0.0016	0.7699 $\pm$ 0.0041	<u>0.7422</u> $\pm$ 0.0046
	RMSE	22.3126 $\pm$ 0.0005	22.3002 $\pm$ 0.0231	19.1291 $\pm$ 7.1169	22.2340 $\pm$ 0.1064	21.9880 $\pm$ 0.1841	18.7906 $\pm$ 3.0783	-	<u>11.9549</u> $\pm$ 0.1394	10.3416 $\pm$ 0.3383
Taxi	ACC	0.9110 $\pm$ 0.0004	0.9030 $\pm$ 0.0003	0.8968 $\pm$ 0.0056	0.9134 $\pm$ 0.0010	0.8224 $\pm$ 0.0659	0.9049 $\pm$ 0.0003	0.8944 $\pm$ 0.0010	<u>0.9225</u> $\pm$ 0.0022	0.9397 $\pm$ 0.0012
	RMSE	0.3708 $\pm$ 0.0000	0.3697 $\pm$ 0.0016	0.3955 $\pm$ 0.0532	0.3708 $\pm$ 0.0000	0.3968 $\pm$ 0.0229	0.2936 $\pm$ 0.0007	-	<u>0.2900</u> $\pm$ 0.0039	0.2835 $\pm$ 0.0077
Taobao	ACC	0.4357 $\pm$ 0.0000	0.6023 $\pm$ 0.0006	0.4530 $\pm$ 0.0146	0.5907 $\pm$ 0.0017	0.4543 $\pm$ 0.0116	0.4570 $\pm$ 0.0099	0.5105 $\pm$ 0.0024	0.5519 $\pm$ 0.0017	0.5975 $\pm$ 0.0029
	RMSE	<u>0.1337</u> $\pm$ 0.0000	<u>0.1337</u> $\pm$ 0.0000	0.1359 $\pm$ 0.0025	0.1336 $\pm$ 0.0000	0.1386 $\pm$ 0.0038	0.2400 $\pm$ 0.1584	-	0.4599 $\pm$ 0.1370	0.5139 $\pm$ 0.0726
StackOF	ACC	0.4250 $\pm$ 0.0000	0.4490 $\pm$ 0.0010	0.3675 $\pm$ 0.1534	0.4532 $\pm$ 0.0009	0.3537 $\pm$ 0.1233	0.4432 $\pm$ 0.0008	0.4357 $\pm$ 0.0002	<u>0.4753</u> $\pm$ 0.0011	0.4770 $\pm$ 0.0015
	RMSE	1.3745 $\pm$ 0.0000	1.3739 $\pm$ 0.0001	2.0898 $\pm$ 1.4551	1.3742 $\pm$ 0.0002	1.3735 $\pm$ 0.0014	1.0397 $\pm$ 0.0021	-	<u>0.9826</u> $\pm$ 0.0154	0.9824 $\pm$ 0.0211
IPTV	ACC	0.3704 $\pm$ 0.2564	0.7314 $\pm$ 0.0011	0.4017 $\pm$ 0.0073	0.6971 $\pm$ 0.0155	OOM	0.4675 $\pm$ 0.0924	0.5168 $\pm$ 0.0005	0.5417 $\pm$ 0.0048	<u>0.7032</u> $\pm$ 0.0016
	RMSE	2.4446 $\pm$ 1.2778	1.8171 $\pm$ 0.0000	2.0845 $\pm$ 0.2323	1.8032 $\pm$ 0.0181	OOM	1.7855 $\pm$ 0.0071	-	<u>1.7478</u> $\pm$ 0.0057	1.7446 $\pm$ 0.0043
MIMIC-II	ACC	0.8023 $\pm$ 0.0142	0.8244 $\pm$ 0.0068	0.6357 $\pm$ 0.1144	0.8372 $\pm$ 0.0176	0.8151 $\pm$ 0.0100	0.8217 $\pm$ 0.0033	0.5628 $\pm$ 0.0068	0.8605 $\pm$ 0.0037	<u>0.8581</u> $\pm$ 0.0059
	RMSE	1.0043 $\pm$ 0.0001	1.0044 $\pm$ 0.0000	1.8085 $\pm$ 1.5960	1.0040 $\pm$ 0.0001	0.9979 $\pm$ 0.0009	0.8587 $\pm$ 0.0014	-	0.8217 $\pm$ 0.0229	<u>0.8330</u> $\pm$ 0.0197

Table 2: Goodness-of-fit evaluation.

Dataset	THP	EIAN-v1	EIAN-v2
Amazon	-2.3118	-1.1715	<u>-1.2387</u>
Retweet	-4.1465	-3.3831	<u>-3.4118</u>
Taxi	-0.2452	0.3970	<u>0.3878</u>
Taobao	-0.5134	<u>-0.0824</u>	0.0746
StackOF	-2.3319	-2.2235	<u>-2.2486</u>
IPTV	<u>-0.3802</u>	-0.6728	-0.3271
MIMIC-II	-6.0089	-4.1322	<u>-4.6834</u>

5.2 Interpretability Analysis

This section evaluate the proposed model’s interpretability of the underlying event interaction patterns from both qualitative and quantitative perspectives under the setting of Section 4.5. Although both EIAN-v1 and EIAN-v2 support this analysis, we consistently use EIAN-v2 to construct cross-attention maps in the following due to its explicit handling of temporal information.

5.2.1 Qualitative Analysis. Since the ground-truth event interaction patterns for real-world datasets are not available, we conduct a qualitative analysis. Figure 2 provides the results on three datasets. The element at the z -th row and z' -th column represents the corresponding element in the normalized cross-attention map $\hat{A}$.

Amazon. Figure 2(a) highlights two key patterns. First, diagonal elements have high weights. Second, the first two rows also have relatively high weights. These two points reflect user behavior patterns well. The former indicates that users tend to review products of the same category, suggesting that once a user has reviewed a product of certain category, they are likely to review similar products in the future. The latter reveals that users who have reviewed products of other categories also tend to review products from the first and second categories. In fact, the two categories represent

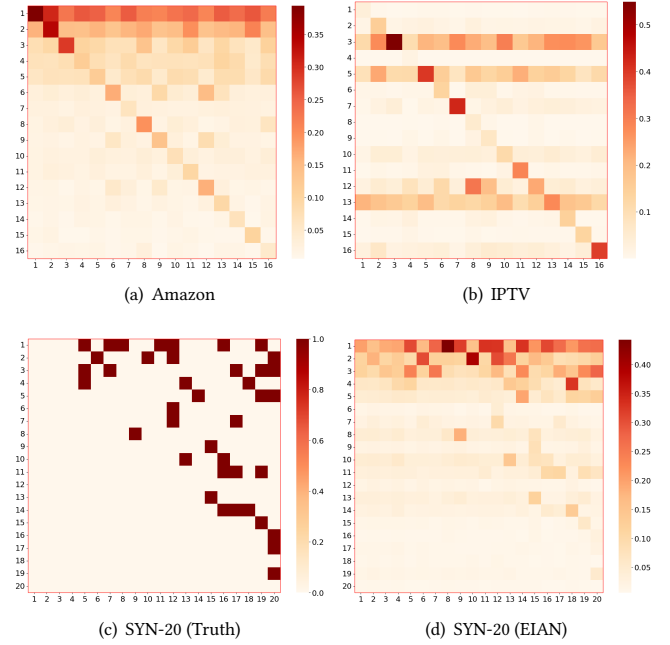


Figure 2: Visualization of the normalized cross-attention maps.

“Clothing” and “Shoes”, two most popular product categories in this dataset.

For another e-commerce dataset, **Taobao**, Figure 4(c) exhibits similar results, reflecting cross-platform similarities among users.

IPTV. Figure 2(b) shows that most diagonal elements have high weights, which means a user tends to watch TV programs of the same categories frequently. Most elements in rows {3,5,13} also

Table 3: Attention map v.s. True event interaction patterns.

Dataset	THP (self-attention)			ELAN (explicit-attention)		
	TPR $\uparrow$	FPR $\downarrow$	AUROC $\uparrow$	TPR $\uparrow$	FPR $\downarrow$	AUROC $\uparrow$
SYN-20	0.9250	0.1889	0.9128	0.9750	0.1639	0.9621
SYN-40	1.0000	0.2980	0.9506	1.0000	0.2645	0.9771
SYN-60	0.9917	0.2503	0.9655	0.9917	0.2227	0.9815
SYN-80	1.0000	0.2237	0.9732	0.9938	0.2136	0.9794
SYN-100	0.9950	0.2410	0.9843	0.9950	0.2247	0.9891

exhibit high weights, where these rows denote “drama”, “entertainment”, and “others”, respectively. This observation highlights similar preferences among most users, reflecting their shared interest in watching programs categorized as “drama” and “entertainment”. In addition, we find some interesting details, such as: (1) programs of category-10 (“movie”) have a significant positive impact on programs of category-3 (“drama”); (2) programs of categories 6 (“finance”), 8 (“laws”), and 9 (“military”) all exhibit a notable influence on programs of category-12 (“news”).

5.2.2 Quantitative Analysis. To conduct further quantitative analysis, we generate several synthetic datasets via the Hawkes process [3]. The generation process involved creating a random graph A (each node represents an event type) with weighted edges to represent event interaction patterns among event types and using it as the excitation matrix for a Hawkes process to generate event sequences. Specially, we name the synthetic datasets based on the number of event types. For example, SYN-20 represents a dataset with 20 types of events. To evaluate the model interpretability, we compare the normalized cross-attention map $\hat{A}$ with the ground-truth graph A using TPR, FPR, and AUROC, which are standard metrics to measure graph similarities [9]. We compared the attention map of our proposed model with the attention map of THP, which employs a standard self-attention. As shown in Table 3, across all datasets with different number of event types (from 20 to 100), our model achieves high TPR and AUROC values with relatively low FPR values, suggesting its strong ability to explain the underlying event interaction patterns among event types. We present the ground-truth event interaction maps and cross-attention maps for SYN-20 in Figure 2(c-d).

5.3 Intrinsic Evaluation

Cross-Attention Settings: In this section, we compare two versions of our model, which differ in the cross-attention module (Section 4.3). As shown in Tables 1 and 2, both versions perform competitively on most datasets, except Amazon and IPTV. ELAN-v1 outperforms on Amazon, whereas ELAN-v2 excels on IPTV. According to statistics (Appendix B.1), Amazon has the narrowest time interval range among these datasets, while IPTV has the widest. Typically, a narrow time interval range indicates less fluctuation in time intervals, making the dataset less sensitive to them, while a wider time interval range has the opposite effect. This suggests that ELAN-v1 suits scenarios less sensitive to time intervals, while ELAN-v2 is more effective for time-interval-sensitive scenarios.

Hyper-Parameter Analysis: Appendix B.2 studies the effect of hyper-parameter settings, including the number of self-attention

Table 4: Results of ablation study.

Dataset	ELAN-v2 w/o enc		ELAN-v2 w/o lin		ELAN-v2	
	ACC $\uparrow$	RMSE $\downarrow$	ACC $\uparrow$	RMSE $\downarrow$	ACC $\uparrow$	RMSE $\downarrow$
Amazon	0.3549	0.3309	0.4346	2.0995	0.4895	0.2648
Retweet	0.5974	15.9322	0.6301	4.6749	0.7422	10.3416
Taxi	0.9156	0.2835	0.9246	0.3677	0.9397	0.2835
Taobao	0.5822	0.9433	0.5516	0.1348	0.5975	0.5139
StackOF	0.4564	1.0216	0.4687	1.3555	0.4770	0.9824
IPTV	0.6809	1.7814	0.6685	1.8137	0.7032	1.7446
MIMIC-II	0.8546	0.8926	0.8453	1.0438	0.8581	0.8330

heads for each temporal encoder block, the number of temporal encoder blocks, the number of query vectors for each event type, the dimension of the initial event embedding, and auxiliary loss weighting factors.

5.4 Ablation Studies

In this section, we conduct ablation studies on two modules of our framework and summarize the result in Table 4. Firstly, we derive a variant, **ELAN-v2 w/o enc**, by discarding the encoder and directly feeding the initial event embeddings (Section 4.2) into the downstream layers. This consistently degrades performance on all datasets, corroborating the necessity of modeling intra-type temporal dynamics. Secondly, we substitute our linear-layer predictor with the intensity-function module adopted in EasyTPP [36], yielding **ELAN-v2 w/o lin**. The resulting performance drop across most datasets is statistically significant. The deterioration can be attributed to the numerical integration and sampling steps inherent to intensity-based formulations, whose performance is bounded by sampling frequency and range. These ablations underscore the critical role of both the intra-type temporal encoder and the linear prediction heads in our design.

6 Conclusion

This paper addresses the challenge of modeling and interpreting the event interactions in event sequences, a critical task for understanding user behavior in online applications. We propose ELAN, an explicit interaction-aware attention network that enhances both flexibility and interpretability in event sequence modeling. By integrating an intra-type temporal encoder and a cross-type interaction decoder, ELAN effectively captures intra-type temporal dynamics and cross-type interactions. Besides, two temporal encoding mechanisms are integrated to improve the model to handle scenarios with diverse inter-event time interval distributions. Extensive experiments demonstrate that ELAN outperforms existing models, providing superior predictive performance and clearer insights into the underlying event interactions.

7 Acknowledge

This work was supported by the Major Program (JD) of Hubei Province (2023BAA024). The computation is completed in the HPC Platform of Huazhong University of Science and Technology.

References

- [1] Massil Achab, Emmanuel Bacry, Stéphane Gaïffas, Iacopo Mastromatteo, and Jean-François Muzy. 2017. Uncovering Causality from Multivariate Hawkes Integrated Cumulants. In *Proceedings of the ICML 2017*. 1–10.
- [2] Arthur Asuncion, David Newman, et al. 2007. UCI machine learning repository.
- [3] Emmanuel Bacry, Martin Bompairé, Stéphane Gaïffas, and Søren Poulsen. 2018. Tick: a Python library for statistical learning, with a particular emphasis on time-dependent modelling. *The Journal of Machine Learning Research* 18, 214 (2018), 1–5.
- [4] Debarun Bhattacharjya, Dharmashankar Subramanian, and Tian Gao. 2018. Proximal graphical event models. In *proceedings of the NeurIPS 2018*. 8147–8156.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *proceedings of the NeurIPS 2020*. 1877–1901.
- [6] Ricky TQ Chen, Brandon Amos, and Maximilian Nickel. 2021. Neural spatio-temporal point processes. In *proceedings of the ICLR 2021*. 1–19.
- [7] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. On the Properties of Neural Machine Translation: Encoder-decoder Approaches. In *Proceedings of SSST-8*. 103–111.
- [8] Nan Du, Hanjun Dai, Rakshit Trivedi, Utkarsh Upadhyay, Manuel Gomez-Rodriguez, and Le Song. 2016. Recurrent marked temporal point processes: Embedding event history to vector. In *proceedings of the KDD 2016*. 1555–1564.
- [9] Chang Gong, Chuzhe Zhang, Di Yao, Jingping Bi, Wenbin Li, and Yongjun Xu. 2024. Causal discovery from temporal data: An overview and new perspectives. *Comput. Surveys* 57, 4 (2024), 1–38.
- [10] Clive WJ Granger. 1969. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica: journal of the Econometric Society* (1969), 424–438.
- [11] Alan G Hawkes. 1971. Spectra of some self-exciting and mutually exciting point processes. *Biometrika* 58, 1 (1971), 83–90.
- [12] S Hochreiter. 1997. Long Short-term Memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [13] Matthew D Hoffman, David M Blei, Chong Wang, and John Paisley. 2013. Stochastic variational inference. *Journal of Machine Learning Research* 14 (2013), 1303–1347.
- [14] Valerie Isham and Mark Westcott. 1979. A self-correcting point process. *Stochastic processes and their applications* 8, 3 (1979), 335–347.
- [15] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical Reparameterization with Gumbel-Softmax. In *proceedings of the ICLR 2016*. 1–13.
- [16] Junteng Jia and Austin R Benson. 2019. Neural jump stochastic differential equations. *Advances in Neural Information Processing Systems* 32 (2019).
- [17] Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. *The annals of mathematical statistics* 22, 1 (1951), 79–86.
- [18] Jure Leskovec and Rok Sosič. 2016. Snap: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology* 8, 1 (2016), 1–20.
- [19] Shuang Li, Shuai Xiao, Shixiang Zhu, Nan Du, Yao Xie, and Le Song. 2018. Learning temporal point processes via reinforcement learning. In *proceedings of the NeurIPS 2018*. 8428–8438.
- [20] Rafael Lima and Jaesik Choi. 2017. Make Hawkes Processes Explainable by Decomposing Self-Triggering Kernels. *arXiv preprint arXiv:1703.09068* (2017).
- [21] Dixon Luo, Hongteng Xu, Yi Zhen, Xia Ning, Hongyuan Zha, Xiaokang Yang, and Wenjun Zhang. 2015. Multi-task multi-dimensional hawkes processes for modeling event sequences. In *proceedings of the IJCAI 2015*. 3685–3691.
- [22] Hongyuan Mei and Jason M Eisner. 2017. The neural hawkes process: A neurally self-modulating multivariate point process. In *proceedings of the NeurIPS 2017*. 6757–6767.
- [23] Zizhuo Meng, Ke Wan, Yadong Huang, Zhidong Li, Yang Wang, and Feng Zhou. 2024. Interpretable Transformer Hawkes Processes: Unveiling Complex Interactions in Social Networks. In *proceedings of the KDD 2024*. 2200–2211.
- [24] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *proceedings of the EMNLP-IJCNLP 2019*. 188–197.
- [25] Yoshihiko Ogata. 1988. Statistical models for earthquake occurrences and residual analysis for point processes. *Journal of the American Statistical association* 83, 401 (1988), 9–27.
- [26] Conny Palm. 1943. Intensitätsschwankungen im fernsprecher. *Ericsson technics* (1943).
- [27] Jie Qiao, Ruichu Cai, Siyu Wu, Yu Xiang, Keli Zhang, and Zhifeng Hao. 2023. Structural Hawkes processes for learning causal structure from discrete-time event sequences. In *proceedings of the IJCAI 2023*. 5702–5710.
- [28] Jakob Gulddahl Rasmussen. 2018. Lecture notes: Temporal point processes and the conditional intensity function. *arXiv preprint arXiv:1806.00221* (2018).
- [29] Mohammed Saeed, Christine Lieu, Greg Raber, and Roger G Mark. 2002. MIMIC II: a massive temporal ICU patient database to support research in intelligent patient monitoring. In *proceedings of the Computers in cardiology*. 641–644.
- [30] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
- [31] A Vaswani. 2017. Attention is all you need. In *proceedings of the NeurIPS 2017*. 6000–6010.
- [32] Chris Whong. 2014. FOILing NYC’s taxi trip data. *FOILing NYCs Taxi Trip Data*. Np 18 (2014), 14.
- [33] Dongxia Wu, Tsuyoshi Idé, Georgios Kollias, Jiri Navratil, Aurelie Lozano, Naoki Abe, Yian Ma, and Rose Yu. 2024. Learning Granger Causality from Instance-wise Self-attentive Hawkes Processes. In *proceedings of the International Conference on Artificial Intelligence and Statistics*. 415–423.
- [34] Shuai Xiao, Junchi Yan, Mehrdad Farajtabar, Le Song, Xiaokang Yang, and Hongyuan Zha. 2019. Learning Time Series Associated Event Sequences With Recurrent Point Process Networks. *IEEE Transactions on Neural Networks and Learning Systems* 30, 10 (2019), 3124–3136.
- [35] Hongteng Xu, Mehrdad Farajtabar, and Hongyuan Zha. 2016. Learning granger causality for hawkes processes. In *proceedings of the ICML 2016*. 1717–1726.
- [36] Siqiao Xue, Xiaoming Shi, Zhixuan Chu, Yan Wang, Hongyan Hao, Fan Zhou, Caigao JIANG, Chen Pan, James Y Zhang, Qingsong Wen, et al. 2024. EasyTPP: Towards Open Benchmarking Temporal Point Processes. In *proceedings of the ICLR 2024*. 1–22.
- [37] Siqiao Xue, Xiaoming Shi, James Y Zhang, and Hongyuan Mei. 2022. HYPRO: a hybridly normalized probabilistic model for long-horizon prediction of event sequences. In *proceedings of the NeurIPS 2022*. 34641–34650.
- [38] Chenghao Yang, Hongyuan Mei, and Jason Eisner. 2022. Transformer embeddings of irregularly spaced events and their participants. In *proceedings of the ICLR 2022*. 1–22.
- [39] Qiang Zhang, Aldo Lipani, Omer Kirnap, and Emine Yilmaz. 2020. Self-attentive Hawkes process. In *proceedings of the ICML 2020*. 11183–11193.
- [40] Shuai Zhang, Chuan Zhou, Yang Aron Liu, Peng Zhang, Xixun Lin, and Zhi-Ming Ma. 2024. Neural Jump-Diffusion Temporal Point Processes. In *proceedings of the ICML 2024*. 60541–60557.
- [41] Wei Zhang, Thomas Panum, Somesh Jha, Prasad Chalasani, and David Page. 2020. Cause: Learning granger causality from event sequences using attribution methods. In *proceedings of the ICML 2020*. 11235–11245.
- [42] Yunhao Zhang and Junchi Yan. 2021. Neural Relation Inference for Multi-dimensional Temporal Point Processes via Message Passing Graph.. In *proceedings of the IJCAI 2021*. 3406–3412.
- [43] Qingyuan Zhao, Murat A Erdogdu, Hera Y He, Anand Rajaraman, and Jure Leskovec. 2015. Seismic: A self-exciting point process model for predicting tweet popularity. In *proceedings of the KDD 2015*. 1513–1522.
- [44] Ke Zhou, Hongyuan Zha, and Le Song. 2013. Learning social infectivity in sparse low-rank networks using multi-dimensional hawkes processes. In *proceedings of the Artificial intelligence and statistics*. 641–649.
- [45] Ke Zhou, Hongyuan Zha, and Le Song. 2013. Learning triggering kernels for multi-dimensional hawkes processes. In *proceedings of the ICML 2013*. 1301–1309.
- [46] Simiao Zuo, Haoming Jiang, Zichong Li, Tuo Zhao, and Hongyuan Zha. 2020. Transformer hawkes process. In *proceedings of the ICML 2020*. 11692–11702.

Appendix

A Recovering event interaction patterns

A.1 Correlation Identification via Cross-Attention Map

We construct a normalized cross-attention map to recover correlations between event types. Given an event sequence with L events, we can define the normalized cross-attention map as follows:

$$\hat{A}_{z,z'} = \frac{\sum_{i=1}^{L-1} \sum_{j=1}^i \mathbb{I}(z_j = z') \mathbb{I}(z_i = z) \mathbb{F}(z', z, j, i+1)}{\sum_{j=1}^{L-1} \mathbb{I}(z_j = z')} \quad (21)$$

The denominator normalizes for varying event type frequencies. $\mathbb{F}(z', z, j, i+1)$ is a function that computes the contribution of a type- z' event at t_j on a type- z event at t_{i+1} . For EIAN-v1 (vanilla query), we can formulate $\mathbb{F}(z', z, j, i+1)$ as follows:

$$\mathbb{F}(z', z, j, i+1) = \frac{S_{dec}(\vec{u}_z, \vec{e}_j)}{\sum_{j=1}^i S_{dec}(\vec{u}_z, \vec{e}_j)} \quad (22)$$

where $S_{dec}(\vec{u}_z, \vec{e}_j)$ follows the same formulation as Eq(8). For EIAN-v2 (time-encoded query), we can formulate $\mathbb{P}(z', z, j, i + 1)$ as follows:

$$\mathbb{P}(z', z, j, i + 1) = \frac{S_{dec}(\vec{u}_z, \vec{e}_j, \Delta t_{ij})}{\sum_{j=1}^i S_{dec}(\vec{u}_z, \vec{e}_j, \Delta t_{ij})} \quad (23)$$

where $S_{dec}(\vec{u}_z, \vec{e}_j, \Delta t_{ij})$ follows the same definition as Eq(13), which explicitly injects inter-event time interval information.

In practice, both (EIAN-v1 and EIAN-v2) support constructing the cross-attention map, for simplicity, we consistently use EIAN-v2 in all interpretability experiments.

A.2 Causal Discovery via Variational Inference

In this section, we integrate causal graphs into EIAN and use Variational Inference [13] to learn unknown causal relationships from event sequences. We also analyze the connection between learned causal links and Granger causality [9] for multi-type events.

A.2.1 Causal-driven Explicit Interaction-aware Network. EIAN's explicit interaction modeling allows direct integration of a causal graph $A = \{A_{z,z'}\}_{z,z'=1}^N$ into cross-attention. If $A_{z,z'} = 1$, the occurrence of a type- z' event can cause the occurrence of a type- z event. We can re-formulate Eq(8) as follows:

$$S_{dec}(\vec{u}_z, \vec{e}_j) = \exp(\phi(\vec{u}_z, \vec{e}_j)) \cdot A_{z,z'} \quad (24)$$

where $S_{dec}(\vec{u}_z, \vec{e}_j)$ represents the unscaled cross-attention score between the type-wise query vector $\vec{u}_z$ and the j -th historical event (assumed to be of type- z'). This score is non-zero if and only if there exists a directed edge $z' \rightarrow z$ in A . Similarly, based on the cross-attention mechanism with time-encoded query, we can also re-formulate Eq(13) as follows:

$$S_{dec}(\vec{u}_z, \vec{e}_j, \Delta t_{ij}) = \exp(\phi(\vec{u}_z, \vec{e}_j, \Delta t_{ij})) \cdot A_{z,z'} \quad (25)$$

By involving the causal graph A into the cross-attention procedure, we can denote the conditional likelihood (Eq(17)) of an observed event sequence S as $p(S|A)$.

A.2.2 Causal Variational Inference Framework. Causal discovery [9] infers a causal graph from event sequences. We combine EIAN with Variational Inference for this task.

The goal is to learn the unknown graph A . We approximate the ground-truth posterior distribution $p(A|S)$ with a variational distribution $q_\phi(A|S)$. And the objective of causal discovery can be further formulated as: minimizing the KL divergence [17] between $q_\phi(A|S)$ and $p(A|S)$. But it is intractable as $p(A|S)$ is unknown. Fortunately, according to the theory of Variational Inference [13], the log-likelihood of observing an event sequence S can be expressed as follows:

$$\log p(S) = \mathcal{L}_{ELBO} + \text{KL}(q_\phi(A|S) \parallel p(A|S)) \quad (26)$$

where the Evidence Lower Bound (ELBO) is:

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q_\phi(A|S)} [\log p_\theta(S|A)] - \text{KL}(q_\phi(A|S) \parallel p(A)) \quad (27)$$

where $\log p_\theta(S|A)$ is the log-likelihood of the event sequence S conditioned on the underlying causal graph A , and θ is our model parameter. $\text{KL}(q_\phi(A|S) \parallel p(A))$ denotes the similarity between

$q_\phi(A|S)$ and $p(A)$. Since S is a non-negative constant, maximizing $\mathcal{L}_{ELBO}$ minimizes the KL divergence to the true posterior.

To learn a single causal graph explaining the generation of all sequences, we assume a global causal graph shared across sequences [42]. Under this assumption, we can equate $q_\phi(A|S)$ to $q_\phi(A)$ and simplify (Eq(27)) as follows:

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q_\phi(A)} [\log p_\theta(S|A)] - \text{KL}(q_\phi(A) \parallel p(A)) \quad (28)$$

We reparameterize $q_\phi(A)$ using Gumbel-Softmax [15] to enable gradient-based optimization. A trainable parameter matrix $\phi \in \mathbb{R}^{N \times N}$ for $q_\phi(A)$ is transformed via sigmoid into a causal probability graph $\mathcal{P} \in \mathbb{R}^{N \times N}$, where $\mathcal{P}_{z',z}$ denotes the probability that there exists a direct edge from event type- z' to event type- z . Subsequently, A graph sample A is drawn as:

$$A_{z',z} = \frac{\exp((\log(\mathcal{P}_{z',z}) + g_{z',z})/\tau)}{\exp((\log(\mathcal{P}_{z',z}) + g_{z',z})/\tau) + \exp((\log(1 - \mathcal{P}_{z',z}) + g_{z',z})/\tau)} \quad (29)$$

where τ is a temperature parameter controlling the smoothness of Gumbel Softmax. $\{g_{z',z}\}_{z',z=1}^N$ are a set of independent and identically distributed samples from a Gumbel(0,1) distribution, with $g = -\log(-\log(u))$ and $u \sim \text{Uniform}(0, 1)$.

The KL divergence term $\text{KL}(q_\phi(A) \parallel p(A))$ in Eq(28) is intractable due to unknown prior $p(A)$. To address this, We instead impose directed acyclic graphs (DAGs) sparsity and acyclicity priors [9] via a loss:

$$\mathcal{L}_{prior_matching} = \lambda_1 * \|\mathcal{P}\|_1 + \lambda_2 * h(\mathcal{P})$$

where λ_1 and λ_2 are hyper-parameters. $\|\cdot\|_1$ is the L1 norm to encourages sparsity, and

$$h(\mathcal{P}) = \text{tr}(e^{\mathcal{P} \odot \mathcal{P}}) - N \quad (30)$$

where $\odot$ is the Hadamard product measures acyclicity ($h(\mathcal{P}) = 0$ for a DAG).

A.2.3 Theoretical Analysis. We connect learned causal relations to Granger causality [10, 41].

Definition I. For event sequences with multiple event types, event type- k' is Granger causal for event type- z if and only if historical events of type- z' contribute to the prediction of subsequent events of type- z . Otherwise, event type- k' is said to be Granger non-causal for event type- z .

Proposition I. For a given event sequence dataset, assuming $\mathcal{P}$ is the causal probability graph learned through maximizing $\mathcal{L}_{ELBO}$ (Eq(28)), then event type z' is Granger non-causal for event type z if and only if $\mathcal{P}_{z,z'}$ converges to 0.

Proof I. During training, the temperature parameter τ of the Gumbel Softmax in Eq(29) is scheduled to gradually reduce, causing the Gumbel Softmax to converge to a strict Bernoulli distribution [15]. Given the causal probability graph $\mathcal{P}$ and assume $\mathcal{P}_{z,z'} = 0$, then the causal graph A sampled from the Bernoulli distribution always satisfies $A_{z,z'} = 0$. This means that the cross-attention score between event type- z 's query vector $\vec{u}_z$ and any historical event of type- z' must be 0. Therefore, the historical events of type- z' do not influence the computation of event type- z 's state representation as defined in Eq (10) and Eq (14), thus do not contribute to

the prediction of the conditional intensity value for event type- z (Eq(17)). Furthermore, according to Definition I, we can deduce that event type- z' is Granger non-causal for event type- z if and only if $\mathcal{P}_{k,k'} = 0$. Similarly, we can prove that event type- z' is Granger causal for event type- z if and only if $\mathcal{P}_{z,z'} = 1$.

A.2.4 Post-Processing. After optimization, we threshold the learned probability graph $\mathcal{P}$ to obtain a binary adjacency matrix $\hat{A}$:

$$\hat{A}_{z',z} = \mathbb{I}(\mathcal{P}_{z',z} > \epsilon) \quad (31)$$

where ϵ is a hyper-parameter controlling the pruning process. Edges with probability $\mathcal{P}_{z',z} \leq \epsilon$ are removed.

B Additional Experiments: Event Modeling

B.1 Dataset Statistics

Table 5 summarizes statistics of the 7 real-world datasets.

B.2 Hyper-parameter Analysis

Figure 3 shows the effect of key hyper-parameters: number of attention heads per encoder block (H), number of encoder blocks (E), number of query vectors per type (Q), and embedding dimension (D).

For H , EIAN-v1 peaks at $H = 4$. EIAN-v2 is stable for $H \leq 4$, but drops slightly at $H = 5$. Regarding parameter E , both versions peaks at $E = 4$, with lower performance at $E = 2$ (underfitting) and $E > 4$ (overfitting). For Q , EIAN-v1 stabilizes with increasing Q , while EIAN-v2 peaks at $Q = 2$. As for D , EIAN-v1 performs best at $D = 128$. EIAN-v2 maintains stable for $D \leq 256$, but degrades at $D = 512$.

Table 7 analyzes auxiliary loss weights: with $\gamma_1 = 1$, we grid-search γ_2 and $\gamma_3 \in \{0.5, 1, 2\}$. The ACC/RMSE trade-off confirms the benefit of multi-task learning, where different objectives (type, time, intensity) are balanced for robust performance.

B.3 Implementation Details

Experiments ran on NVIDIA Tesla V100 GPUs (32 GB) with Ubuntu 18.04 and CUDA 11.8. The parameter settings of EIAN for each real-world dataset are provided in the ".yaml" files located in the "./model/configs" directory of the code repository.

B.4 Cross-attention Maps

Figure 4 shows normalized cross-attention maps for real-world datasets. Element at (i,j) represents the average influence weight from event type- j to event type- i . Beyond Amazon, Taobao, and IPTV datasets analyzed in Section 5.2.1, we also observe some interesting phenomena on Retweet, Taxi, and MIMIC-II:

Retweet. In Figure 4(a), row weights (2nd > 3rd > 1st) indicate middle users are most influenced by others' retweets, large users least. Column weights (1st lower, 3rd higher) show small users have minimal influence, while large users have the most, consistent with follower-count dynamics.

Taxi. In Figure 4(b), rows {4,9} ("drop-off" and "pickup" in Manhattan) show high weights. Given Manhattan's high activity share (>42% drop-off, 44% pickup [23]), events in other boroughs often lead to subsequent Manhattan events.

StackOF. In Figure 4(d), weights are mostly low. High weights in rows {4,6,9} (badges: "Popular Question", "Nice Answer", "Notable Question") suggest badge recipients are more likely to earn these specific badges later, aligning with the platform's incentive design.

MIMIC-II. In Figure 4(e), The map is notably sparser compared to the results on other datasets, indicating limited inter-type influence. This likely stems from the dataset's characteristics (Table 5): most event types, but few short sequences, capturing only sparse interaction patterns.

Figure 2(c), 2(d) and 5 show ground-truth and learned maps for three synthetic datasets. The close alignment demonstrates EIAN's capacity to recover true interaction patterns.

B.5 Results on time-interval-sensitive datasets

We test on five synthetic Hawkes process datasets (20-100 event types), where influence decays exponentially with time, making them interval-sensitive. Table 8 shows EIAN-v2 (with rotary encoding) outperforms EIAN-v1 in ACC across all settings, confirming its effectiveness for interval-sensitive scenarios.

C Additional Experiments: Causal Discovery for Event Sequences

C.1 Baseline Models

For the causal discovery task, we compare the proposed model (Appendix A.2) with seven baselines: **HEXP** [11], **ADM4** [44], **NPHC** [1], **RPPN** [34], **CAUSE** [41], **SHP** [27] and **ISAHP** [33].

C.2 Dataset Statistics

We use two benchmark datasets: **Diabetes** [2] (real-world, daily events of 70 patients) and **PGEM** [4] (synthetic, generated via Proximal Graphical Event Model).

C.3 Results for Causal Discovery

We use five metrics: True Positive Rate (TPR), False Positive Rate (FPR), F1 score (F1), Area Under the Receiver Operating Characteristic Curve (AUROC), and Kendall's τ . Results in Table 6 show: **Diabetes:** Both EIAN versions significantly outperform baselines, showing high TPR and low FPR. **PGEM:** ADM4 has best F1, but EIAN-v2 achieves highest AUROC and Kendall's τ , indicating strong ranking reliability. Overall, EIAN proves to be a reliable and generalizable solution for causal discovery.

C.4 Limitation and Future Work

EIAN enhances predictive performance and interpretability. Limitations and future directions include:

Global vs. Local Causal Graphs: For causal discovery, we assumed a shared global graph due to dataset constraints, yielding a coarse-grained view. Learning sequence-specific causal graphs could offer finer-grained insights.

Application to Recommendation: Event sequence modeling methods could be adapted for recommendation tasks in the future.

Table 5: Statistics of the real-world datasets.

Dataset	N	Data Split (Sequence)			Data Split (Event)			Sequence Length			Inter-Event Time Interval		
		Train	Val	Test	Train	Val	Test	Min	Max	Avg	Min	Max	Avg
Amazon[24]	16	6454	922	1851	288377	40995	84048	14	94	45	0.01	0.80	0.52
Retweet[45]	3	9000	1535	1520	369731	62823	61154	10	97	41	4.27×10^{-8}	100.00	13.31
Taxi[32]	10	1400	200	400	51854	7404	14820	36	38	37	2.78×10^{-4}	5.72	0.22
Taobao[37]	17	1300	200	500	75205	11737	28455	32	64	58	0.00	2.00	0.18
StackOF[18]	22	1401	401	401	90497	25762	26518	41	101	65	0.00	20.34	0.86
IPTV[21]	16	1302	279	279	678476	140766	147057	11	1000	520	0.00	156.90	0.29
MIMIC-II[29]	75	527	58	65	1930	252	237	2	23	4	0.02	5.62	0.64

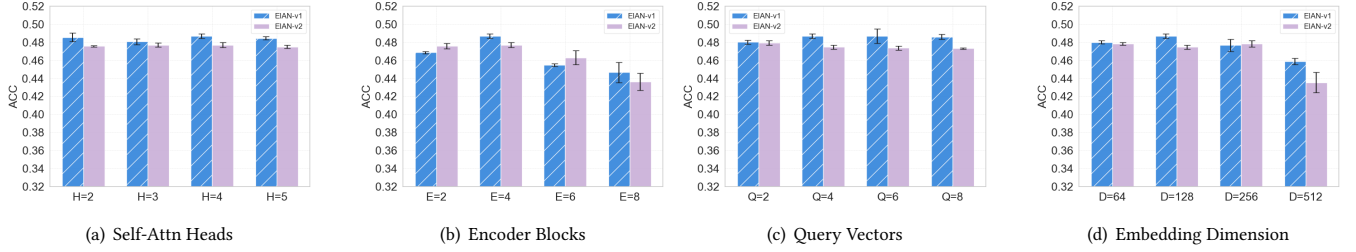


Figure 3: Hyper-parameter analysis on the StackOF dataset.

Table 6: Performance comparison about Causal Discovery on the Diabetes and PGEM datasets.

Model	Diabetes					PGEM				
	TPR $\uparrow$	FPR $\downarrow$	F1 $\uparrow$	AUROC $\uparrow$	Kendall's $\tau \uparrow$	TPR $\uparrow$	FPR $\downarrow$	F1 $\uparrow$	AUROC $\uparrow$	Kendall's $\tau \uparrow$
HExp	0.3636	0.1772	0.1860	0.6007	0.0711	1.0000	0.3085	0.2927	0.9557	0.3079
ADM4	0.1818	0.1772	0.0976	0.5567	0.0451	1.0000	0.0319	0.8000	0.9787	0.3255
NPHC	N/A	N/A	N/A	N/A	N/A	0.5000	0.4043	0.1277	0.5053	0.0036
RPPN	0.6364	0.5127	0.1414	0.4856	-0.0101	0.8333	0.7128	0.1282	0.8298	0.2226
CAUSE	0.4545	0.2595	0.1754	0.6720	0.1204	1.0000	0.7234	0.1500	0.9255	0.2873
SHP	0.2727	0.1582	0.1538	0.5506	0.0641	0.8333	0.3723	0.2173	0.8182	0.2680
ISAFP	0.7273	0.5886	0.1429	0.5693	0.0698	1.0000	0.2979	0.3000	0.9362	0.2945
EIAN-v1	0.7273	<u>0.1456</u>	<u>0.3810</u>	<u>0.8291</u>	<u>0.2309</u>	1.0000	0.0851	0.6000	<u>0.9929</u>	<u>0.3343</u>
EIAN-v2	0.7273	0.1329	0.4000	0.8446	0.2418	1.0000	<u>0.0426</u>	<u>0.7500</u>	1.0000	0.3391

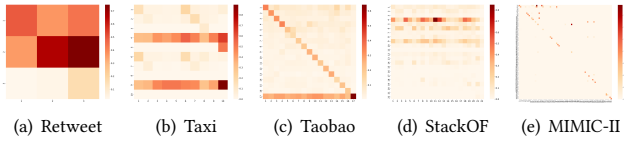


Figure 4: Visualization of the normalized cross-attention maps on four representative real-world datasets.

Table 7: Loss factors analysis on Amazon dataset(ACC/RMSE)

		γ_2		
		0.5	1	2
γ_3	0.5	51.41%/0.2925	50.93%/0.2860	51.40%/0.2813
	1	50.67%/0.2974	51.92%/0.2891	52.07%/0.2860
	2	50.81%/0.3062	50.37%/0.2966	51.71%/0.2939

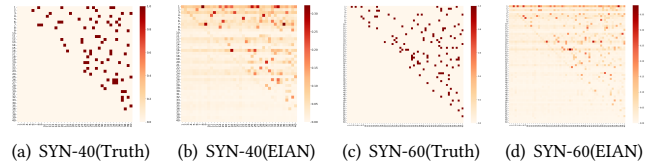


Figure 5: Visualization of the normalized cross-attention maps on two representative synthetic datasets.

Table 8: Performance comparison of two temporal encoding mechanisms on five synthetic datasets.

#Types	20	40	60	80	100
EIAN-v1	0.3617	0.2559	0.2625	0.2540	0.1903
EIAN-v2	0.4688	0.4013	0.4202	0.4327	0.3688